

PUC MINAS

Bacharelado em Engenharia de Software



Relatório Técnico: Implementando Padrões de Teste (Test Patterns)

Nome: João Pedro Aguiar do Prado

Matrícula: 756491

Disciplina: Teste de Software

1. Padrões de Criação de Dados (Builders)

Justificativa: `CarrinhoBuilder` vs. `CarrinhoMother`

A escolha do padrão Data Builder (`CarrinhoBuilder`) sobre o Object Mother para a entidade `Carrinho` justifica-se pela complexidade inerente a este objeto. O `Carrinho` pode assumir múltiplos estados (vazio, com diferentes utilizadores, com variados itens e valores). A aplicação de um Object Mother exigiria a criação estática de inúmeros métodos para cobrir todas as permutações necessárias aos testes, resultando numa API inflada e rígida. O padrão Builder, por sua vez, resolve esta limitação ao oferecer uma API fluente e expressiva (através de métodos como `.comUser()`, `.comItens()`, `.vazio()`), permitindo a composição customizada apenas dos dados relevantes para cada cenário específico.

Exemplo de Refatoração (Antes e Depois)

Antes (Setup Complexo e Poluído): A ausência de um padrão resulta no *Test Smell* de Setup Obscuro (Obscure Setup), onde a lógica de instanciação ofusca o verdadeiro propósito do teste.

JavaScript

```
test('cliente premium finaliza compra - manual setup poluído', async () => {
```

JavaScript

```
// Setup Obscuro: Detalhes de implementação expostos no teste
const user = {
  id: 'u1', nome: 'Ana', email: 'ana@x.com', tipo: 'PREMIUM',
  isPremium() { return true; }
};

const itens = [
  { nome: 'Produto 1', preco: 100, sku: 'p1' },
  { nome: 'Produto 2', preco: 100, sku: 'p2' }
];

const carrinho = new Carrinho(user, itens);

// Act & Assert...
});
```

Depois (Aplicação do Data Builder): O setup torna-se explícito e focado exclusivamente no que é importante para o cenário.

JavaScript

```
test('cliente premium finaliza compra - usando CarrinhoBuilder', async () =>
```

```
{
```

JavaScript

```
// Setup limpo, fluente e focado na intenção
const carrinho = new CarrinhoBuilder()
  .comUser(UserMother.umUsuarioPremium())
  .comItens([
    { nome: 'Produto 1', preco: 100 },
    { nome: 'Produto 2', preco: 100 }
  ])
  .build();

// Act & Assert...
});
```

Benefícios Alcançados

A implementação do Data Builder melhora substancialmente a legibilidade do código de teste, tornando a intenção clara e removendo a poluição visual. Adicionalmente, facilita a manutenção a longo prazo: qualquer alteração na estrutura interna da classe `Carrinho` exigirá modificações apenas dentro do construtor do `CarrinhoBuilder`, blindando os testes unitários contra alterações de domínio e prevenindo Testes Frágeis (Fragile Tests).

2. Padrões de Test Doubles (Mocks vs. Stubs)

Análise do Cenário de Teste "Sucesso Premium"

No cenário que valida a finalização de compra por um cliente Premium, as dependências do `CheckoutService` foram substituídas por Test Doubles para garantir o isolamento da unidade testada.

- **Stub:** O `GatewayPagamento` foi instanciado como um Stub.
- **Mock:** O `EmailService` foi instanciado como um Mock.

Diferenciação Tática: Estado vs. Comportamento

A distinção entre a utilização de um Stub e de um Mock recai sobre os objetivos de verificação de cada dependência:

- **GatewayPagamento (Stub e Verificação de Estado):** O Stub fornece respostas enlatadas (canned answers) pré-definidas para permitir que o teste prossiga. Neste cenário, o sistema necessita saber que o pagamento foi processado com sucesso para continuar a sua lógica. O `GatewayPagamento` atua primordialmente fornecendo o estado `{ success: true }` de forma determinística, permitindo focar na verificação de estado do `CheckoutService`.
- **EmailService (Mock e Verificação de Comportamento):** O envio de um e-mail é um efeito colateral cujo estado interno não afeta o fluxo subsequente da função, mas o facto de ter sido invocado é um requisito de negócio indispensável. Portanto, o `EmailService` atua como um Mock para efetuar a Verificação de Comportamento (Behavior Verification). O objetivo não é obter dados de retorno desta dependência, mas sim garantir que houve a interação correta (número de chamadas e argumentos corretos, como o e-mail do cliente e a mensagem adequada).

3. Conclusão

A aplicação deliberada de Padrões de Teste é o mecanismo preventivo primário contra *Test Smells*. A utilização de *Builders* e *Object Mothers* ataca diretamente o Setup Obscuro (Obscure Setup) encapsulando lógicas complexas de instanciação em estruturas reutilizáveis. Paralelamente, o emprego correto de *Test Doubles* garante o isolamento absoluto da unidade de software , eliminando o acoplamento excessivo e a fragilidade das suítes de teste (Fragile Tests). Em suma, a adoção destas práticas transita o foco do programador de meramente "escrever testes" para a "engenharia de testes arquiteturalmente sustentáveis e robustos".